



Contenido

UNIDAD 8 Bloque II. Desarrollo de aplicaciones en Laravel.....	2
Controladores	2
Controlador básico	2
Crear un nuevo controlador.....	3
Controladores y espacios de nombres.....	3
Generar una URL a una acción	4
Controladores implícitos	4
Caché de rutas	5
Middleware o filtros.....	5
Definir un nuevo <i>Middleware</i>	6
<i>Middleware</i> antes o después de la petición.....	7
Uso de Middleware.....	7
Formularios	13
Crear formularios.....	14
Protección contra CSRF	15
Elementos de un formulario.....	15



UNIDAD 8 Bloque II. Desarrollo de aplicaciones en Laravel

Controladores

Hasta el momento hemos visto solamente como devolver una cadena para una ruta y como asociar una vista a una ruta directamente en el fichero de rutas. Pero en general la forma recomendable de trabajar será asociar dichas rutas a un método de un controlador. Esto nos permitirá separar mucho mejor el código y crear clases (controladores) que agrupen toda la funcionalidad de un determinado recurso. Por ejemplo, podemos crear un controlador para gestionar toda la lógica asociada al control de usuarios o cualquier otro tipo de recurso.

Como ya vimos en la sección de introducción, los controladores son el punto de entrada de las peticiones de los usuarios y son los que deben contener toda la lógica asociada al procesamiento de una petición, encargándose de realizar las consultas necesarias a la base de datos, de preparar los datos y llamar a la vista correspondiente con dichos datos.

Controlador básico

Los controladores se almacenan en ficheros PHP en la carpeta `app/Http/Controllers` y normalmente se les añade el sufijo `Controller`, por ejemplo `UserController.php` o `MoviesController.php`. A continuación se incluye un ejemplo básico de un controlador almacenado en el fichero `app/Http/Controllers/UserController.php`:

```
<?php
namespace App\Http\Controllers;
use App\User;
use App\Http\Controllers\Controller;
class UserController extends Controller
{
    /**
     * Mostrar información de un usuario.
     * @param int $id
     * @return Response
     */
    public function showProfile($id)
    {
        $user = User::findOrFail($id);
        return view('user.profile', ['user' => $user]);
    }
}
```

Todos los controladores tienen que extender la clase base `Controller`. Esta clase viene ya creada por defecto con la instalación de Laravel, la podemos encontrar en la carpeta `app/Http/Controllers`. Se utiliza para centralizar toda la lógica que se vaya a utilizar de forma compartida por los controladores de nuestra aplicación. Por defecto solo carga código para validación y autorización, pero podemos añadir en la misma todos los



métodos que necesitemos.

En el código de ejemplo, el método `showProfile($id)` lo único que realiza es obtener los datos de un usuario, generar la vista `user.profile` a partir de los datos obtenidos y devolverla como valor de retorno para que se muestre por pantalla.

Una vez definido un controlador ya podemos asociarlo a una ruta. Para esto tenemos que modificar el fichero de rutas `routes.php` de la forma:

```
Route::get('user/{id}', [UserController::class, 'showProfile']);
```

En lugar de pasar una función como segundo parámetro, tenemos que escribir una cadena que contenga el nombre del controlador, seguido de una arroba `@` y del nombre del método que queremos asociar. No es necesario añadir nada más, ni los parámetros que recibe el método en cuestión, todo esto se hace de forma automática.

Crear un nuevo controlador

Como hemos visto los controladores se almacenan dentro de la carpeta `app/Http/Controllers` como ficheros PHP. Para crear uno nuevo bien lo podemos hacer a mano y rellenar nosotros todo el código, o podemos utilizar el siguiente comando de Artisan que nos adelantará todo el trabajo:

```
php artisan make:controller MoviesController
```

Este comando creará el controlador `MoviesController` dentro de la carpeta `app/Http/Controllers` y lo completará con el código básico que hemos visto antes. Al añadir la opción `--plain` le indicamos que no añada ningún método al controlador, por lo que el cuerpo de la clase estará vacío. De momento vamos a utilizar esta opción para añadir nosotros mismos los métodos que necesitemos. Más adelante, cuando hablemos sobre controladores tipo RESTful, volveremos a ver esta opción.

Controladores y espacios de nombres

También podemos crear sub-carpetas dentro de la carpeta `Controllers` para organizarnos mejor. En este caso, la estructura de carpetas que creamos no tendrá nada que ver con la ruta asociada a la petición y, de hecho, a la hora de hacer referencia al controlador únicamente tendremos que hacerlo a través de su espacio de nombres.

Como hemos visto al referenciar el controlador en el fichero de rutas únicamente tenemos que indicar su nombre y no toda la ruta ni el espacio de nombres `App\Http\Controllers`. Esto es porque el servicio encargado de cargar las rutas añade automáticamente el espacio de nombres raíz para los controladores. Si metemos todos nuestros controladores dentro del mismo espacio de nombres no tendremos que añadir nada más. Pero si decidimos crear sub-carpetas y organizar nuestros controladores en sub-espacios de nombres, entonces sí que tendremos que añadir esa parte.



Por ejemplo, si creamos un controlador en:

```
Use App\Http\Controllers\CatalogController;
```

, entonces para registrar una ruta hasta dicho controlador tendríamos que hacer:

```
Route::get('catalog', [CatalogController::class, 'index']);
```

Generar una URL a una acción

Para generar la URL que apunte a una acción de un controlador podemos usar el método `action` de la forma:

```
$url = action(FooController::class, 'method');
```

Por ejemplo, para crear en una plantilla con *Blade* un enlace que apunte a una acción haríamos:

```
<a href="{{ action(FooController::class, 'method') }}"> ¡Aprieta aquí!  
</a>
```

Controladores implícitos

Laravel también permite definir fácilmente la creación de controladores como recursos que capturen todas las rutas de un determinado dominio. Por ejemplo, capturar todas las consultas que se realicen a la URL "users" o "users" seguido de cualquier cosa (por ejemplo "users/profile"). Para esto en primer lugar tenemos que definir la ruta en el fichero de rutas usando `Route::controller` de la forma:

```
Route::controller('users', 'UserController');
```

Esto quiere decir que todas las peticiones realizadas a la ruta "users" o subrutas de "users" se redirigirán al controlador `UserController`. Además se capturarán las peticiones de cualquier tipo, ya sean GET o POST, a dichas rutas. Para gestionar estas rutas en el controlador tenemos que seguir un patrón a la hora de definir el nombre de los métodos: primero tendremos que poner el tipo de petición y después la sub-ruta a la que debe de responder. Por ejemplo, para gestionar las peticiones tipo GET a la URL "users/profile" tendremos que crear el método "getProfile". La única excepción a este caso es "Index" que se referirá a las peticiones a la ruta raíz, por ejemplo "getIndex" gestionará las peticiones GET a "users". A continuación se incluye un ejemplo:

```
class UserController extends BaseController  
{  
    public function getIndex()  
    {  
        //  
    }  
  
    public function postProfile()  
    {  
    }  
}
```



```
//  
}  
  
public function anyLogin()  
{  
    //  
}}
```

Además, si queremos crear rutas con varias palabras lo podemos hacer usando la notación "*CamelCase*" en el nombre del método. Por ejemplo el método "getAdminProfile" será parseado a la ruta "users/admin-profile".

También podemos definir un método especial que capture las todas las peticiones "perdidas" o no capturadas por el resto de métodos. Para esto simplemente tenemos que definir un método con el nombre `missingMethod` que recibirá por parámetros la ruta y los parámetros de la petición:

```
public function missingMethod($parameters = array())  
{  
    //  
}
```

Caché de rutas

Si definimos todas nuestras rutas para que utilicen controladores podemos aprovechar la nueva funcionalidad para crear una caché de las rutas. Es importante que estén basadas en controladores porque si definimos respuestas directas desde el fichero de rutas (como vimos en el capítulo anterior) la caché no funcionará.

Gracias a la caché Laravel indican que se puede acelerar el proceso de registro de rutas hasta 100 veces. Para generar la caché simplemente tenemos que ejecutar el comando de *Artisan*:

```
php artisan route:cache
```

Si creamos más rutas y queremos añadirlas a la caché simplemente tenemos que volver a lanzar el mismo comando. Para borrar la caché de rutas y no generar una nueva caché tenemos que ejecutar:

```
php artisan route:clear
```

Nota: La caché se recomienda crearla solo cuando ya vayamos a pasar a producción nuestra web. Cuando estamos trabajando en la web es posible que añadamos nuevas rutas y sino nos acordamos de regenerar la caché la ruta no funcionará.

Middleware o filtros

Los componentes llamados *Middleware* son un mecanismo proporcionado por Laravel para **filtrar las peticiones HTTP** que se realizan a una aplicación. Un filtroo *middleware* se



define como una clase PHP almacenada en un fichero dentro de la carpeta `app/Http/Middleware`. Cada *middleware* se encargará de aplicar un tipo concreto de filtro y de decidir que realizar con la petición realizada: permitir su ejecución, dar un error o redireccionar a otra página encaso de no permitirla.

Laravel incluye varios filtros por defecto, uno de ellos es el encargado de realizar la autenticación de los usuarios. Este filtro lo podemos aplicar sobre una ruta, un conjunto de rutas o sobre un controlador en concreto. Este *middleware* se encargará de filtrar las peticiones a dichas rutas: en caso de estar logueado y tener permisos de acceso le permitirá continuar con la petición, y en caso de no estar autenticado lo redireccionará al formulario de login.

Laravel incluye *middleware* para gestionar la autenticación, el modo mantenimiento, la protección contra CSRF, y algunos mas. Todos estos filtros los podemos encontrar en la carpeta `app/Http/Middleware`, los cuales los podemos modificar o ampliar su funcionalidad. Pero además de estos podemos crear nuestros propios *Middleware* como veremos a continuación.

Definir un nuevo *Middleware*

Para crear un nuevo *Middleware* podemos utilizar el comando de Artisan:

```
php artisan make:middleware MyMiddleware
```

Este comando creará la clase `MyMiddleware` dentro de la carpeta `app/Http/Middleware` con el siguiente contenido por defecto:

```
<?php
namespace App\Http\Middleware; use Closure;
class MyMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param \Illuminate\Http\Request $request
     * @param \Closure $next
     * @return mixed
     */
    public function handle($request, Closure $next)
    {
        return $next($request);
    }
}
```

El código generado por Artisan ya viene preparado para que podamos escribir directamente la implementación del filtro a realizar dentro de la función `handle`. Como podemos ver, esta función solo incluye el valor de retorno con una llamada a `return $next($request);` que lo que hace es continuar con la petición y ejecutar el método que tiene que procesarla. Como entrada la función `handle` recibe dos parámetros:



`$request`: En la cual nos vienen todos los parámetros de entrada de la petición.

`$next`: El método o función que tiene que procesar la petición.

Por ejemplo podríamos crear un filtro que redirija al home si el usuario tiene menos de 18 años y en otro caso que le permita acceder a la ruta:

```
public function handle($request, Closure $next)
{
    if ($request->input('age') < 18) {return redirect('home');}
    return $next($request);
}
```

Como hemos dicho antes, podemos hacer tres cosas con una petición:

Si todo es correcto permitir que la petición continúe devolviendo:

```
return $next($request);
```

Realizar una redirección a otra ruta para no permitir el acceso con:

```
return redirect('home');
```

Lanzar una excepción o llamar al método abort para mostrar una página de error:

```
abort(403, 'Unauthorized action.');
```

Middleware antes o después de la petición

Para hacer que el código de un *Middleware* se ejecute antes o después de la petición HTTP simplemente tenemos que poner nuestro código antes o después de la llamada a `$next($request)`. Por ejemplo, el siguiente *Middleware* realizaría la acción **antes** de la petición:

```
public function handle($request, Closure $next)
{
    // Código a ejecutar antes de la petición
    return $next($request);
}
```

Mientras que el siguiente *Middleware* ejecutaría el código **después** de la petición:

```
public function handle($request, Closure $next)
{
    $response = $next($request);
    // Código a ejecutar después de la petición
    return $response;
}
```

Uso de Middleware

De momento hemos visto para que vale y como se define un *Middleware*, en esta sección



veremos cómo utilizarlos. Laravel permite la utilización de *Middleware* de tres formas distintas: global, asociado a rutas o grupos de rutas, o asociado a un controlador o a un método de un controlador. En los tres casos será necesario registrar primero el *Middleware* en la clase `app/Http/Kernel.php`.

Middleware global

Para hacer que un *Middleware* se ejecute con **todas** las peticiones HTTP realizadas a una aplicación simplemente lo tenemos que registrar en el array

`$middleware` definido en la clase `app/Http/Kernel.php`. Por ejemplo:

```
protected $middleware = [
    \Illuminate\Foundation\Http\Middleware\CheckForMaintenanceMode::class,
    \App\Http\Middleware\EncryptCookies::class,
    \Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse::class,
    \Illuminate\Session\Middleware\StartSession::class,
    \Illuminate\View\Middleware\ShareErrorsFromSession::class,
    \App\Http\Middleware\VerifyCsrfToken::class,
    \App\Http\Middleware\MyMiddleware::class,
];
```

En este ejemplo hemos registrado la clase *MyMiddleware* al final del array. Si queremos que nuestro *middleware* se ejecute antes que otro filtro simplemente tendremos que colocarlo antes en la posición del array.

Middleware asociado a rutas

En el caso de querer que nuestro *middleware* se ejecute solo cuando se llame a una ruta o a un grupo de rutas también tendremos que registrarlo en el fichero `app/Http/Kernel.php`, pero en el array `$routeMiddleware`. Al añadirlo a este array además tendremos que asignarle un nombre o clave, que será el que después utilizaremos asociarlo con una ruta.

En primer lugar añadimos nuestro filtro al array y le asignamos el nombre `"es_mayor_de_edad"`:

```
protected $routeMiddleware = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,
    'es_mayor_de_edad' => \App\Http\Middleware\MyMiddleware::class,
];
```

Una vez registrado nuestro *middleware* ya lo podemos utilizar en el fichero de rutas `app/Http/routes.php` mediante la clave o nombre asignado, por ejemplo:

```
Route::get('dashboard', ['middleware' => 'es_mayor_de_edad', function
() {
//...
```




```
});
```

En el ejemplo anterior hemos asignado el *middleware* con *clave es_mayor_de_edad* a la ruta *dashboard*. Como se puede ver se utiliza un array como segundo parámetro, en el cual indicamos el *middleware* y la acción. Si la petición supera el filtro entonces se ejecutará la función asociada.

Para asociar un filtro con una ruta que utiliza un método de un controlador se realizaría de la misma manera pero indicando la acción mediante la clave "uses":

```
Route::get('profile', ['middleware'=>'auth',  
'uses'=> [UserController::class, 'showProfile']]);
```

Si queremos asociar varios *middleware* con una ruta simplemente tenemos que añadir un array con las claves. Los filtros se ejecutarán en el orden indicado en dicho array:

```
Route::get('dashboard', ['middleware' => ['auth', 'es_mayor_de_edad'],  
function () {  
//...  
}]);
```

Laravel también permite asociar los filtros con las rutas usando el método *middleware()* sobre la definición de la ruta de la forma:

```
Route::get('/', function () {  
// ...  
})->middleware(['first', 'second']);  
  
// O sobre un controlador:  
  
Route::get('profile', 'UserController@showProfile')->  
middleware('auth');
```

Middleware dentro de controladores

También es posible indicar el *middleware* a utilizar desde dentro de un controlador. En este caso los filtros también tendrán que estar registrados en el array *\$routeMiddleware* del fichero *app/Http/Kernel.php*. Para utilizarlos se recomienda realizar la asignación en el constructor del controlador y asignar los filtros usando su clave mediante el método *middleware*. Podremos indicar que se filtren todos los métodos, solo algunos, o todos excepto los indicados, por ejemplo:

```
class UserController extends Controller  
{  
/**  
 * Instantiate a new UserController instance.  
 *  
 * @return void  
 */  
public function __construct()  
{  
// Filtrar todos los métodos
```



```
$this->middleware('auth');

// Filtrar solo estos métodos...
$this->middleware('log', ['only' => ['fooAction', 'barAction']]);

// Filtrar todos los métodos excepto...
$this->middleware('subscribed', ['except' => ['fooAction',
'barAction']]);
}
}
```

Revisar los filtros asignados

Al crear una aplicación Web es importante asegurarse de que todas las rutas definidas son correctas y que las partes privadas realmente están protegidas. Para esto Laravel incluye el siguiente método de Artisan:

```
php artisan route:list
```

Este método muestra una tabla con todas las rutas, métodos y acciones. Además para cada ruta indica los filtros asociados, tanto si están definidos desde el fichero de rutas como **desde dentro de un controlador**. Por lo tanto es muy útil para comprobar que todas las rutas y filtros que hemos definido se hayan creado correctamente.

Paso de parámetros

Un *Middleware* también puede recibir parámetros. Por ejemplo, podemos crear un filtro para comprobar si el usuario logueado tiene un determinado rol indicado por parámetro. Para esto lo primero que tenemos que hacer es añadir un tercer parámetro a la función *handle* del *Middleware*:

```
<?php

namespace App\Http\Middleware; use Closure;
class RoleMiddleware
{
    /**
     * Run the request filter.
     *
     * @param \Illuminate\Http\Request $request
     * @param \Closure $next
     * @param string $role
     * @return mixed
     */
    public function handle($request, Closure $next, $role)
    {
        if (! $request->user()->hasRole($role)) {
            // No tiene el rol esperado!
        }

        return $next($request);
    }
}
```



```
}  
  
}
```

En el código anterior de ejemplo se ha añadido el tercer parámetro `$role` a la función. Si nuestro filtro necesita recibir más parámetros simplemente tendríamos que añadirlos de la misma forma a esta función.

Para pasar un parámetro a un *middleware* en la definición de una ruta lo tendremos que añadir a continuación del nombre del filtro separado por dos puntos, por ejemplo:

```
Route::put('post/{id}', ['middleware' => 'role:editor', function ($id)  
{  
//  
}]);
```

Si tenemos que pasar más de un parámetro al filtro los separaremos por comas, por ejemplo: `role:editor,admin`.

Rutas avanzadas

Laravel permite crear grupos de rutas para especificar opciones comunes a todas ellas, como por ejemplo un *middleware*, un prefijo, un subdominio o un espacio de nombres que se tiene que aplicar sobre todas ellas.

A continuación vamos a ver algunas de estas opciones, en todos los casos usaremos el método `Route::group`, el cual recibirá como primer parámetro las opciones a aplicar sobre todo el grupo y como segundo parámetro una cláusula con la definición de las rutas.

Middleware sobre un grupo de rutas

Esta opción es muy útil para aplicar un filtro sobre todo un conjunto de rutas, de esta forma solo tendremos que especificar el filtro una vez y además nos permitirá dividir las rutas en secciones (distinguiendo mejor a que secciones se les está aplicando un filtro):

```
Route::group(['middleware' => 'auth'], function () { Route::get('/',  
function () {  
// Ruta filtrada por el middleware  
});
```

```
Route::get('user/profile', function () {  
  
// Ruta filtrada por el middleware  
  
});  
  
});
```

Grupos de rutas con prefijo

También podemos utilizar la opción de agrupar rutas para indicar un prefijo que se

añadirá a todas las URL del grupo. Por ejemplo, si queremos definir una sección de rutas que empiecen por el prefijo `dashboard` tendríamos que hacer lo siguiente:

```
Route::group(['prefix' => 'dashboard'], function () {
Route::get('catalog', function () { /* ... */ });
Route::get('users', function () { /* ... */ });
});
```

También podemos crear grupos de rutas dentro de otros grupos. Por ejemplo para definir un grupo de rutas a utilizar en una API y crear diferentes rutas según la versión de la API podríamos hacer:

```
Route::group(['prefix' => 'api'], function()
{
Route::group(['prefix' => 'v1'], function()
{
});
// Rutas con el prefijo api/v1
Route::get('recurso', [CatalogAPIv1::class, 'getRecurso']);
Route::post('recurso', [CatalogAPIv1::class, 'postRecurso']);
Route::get('recurso/{id}', [CatalogAPIv1::class, 'putRecurso']);
Route::group(['prefix' => 'v2'], function()
{
});
}); // Rutas con el prefijo api/v2
Route::get('recurso', [CatalogAPIv2::class, 'getRecurso']);
Route::post('recurso', [CatalogAPIv2::class, 'postRecurso']);
Route::get('recurso/{id}', [CatalogAPIv2::class, 'putRecurso']);
```

De esta forma podemos crear secciones dentro de nuestro fichero de rutas para agrupar, por ejemplo, todas las rutas públicas, todas las de la sección privada de administración, sección privada de usuario, las rutas de las diferentes versiones de la API de nuestro sitio, etc.

Esta opción también la podemos aprovechar para especificar parámetros comunes que se recogerán para todas las rutas y se pasarán a todos los controladores o funciones asociadas, por ejemplo:

```
Route::group(['prefix' => 'accounts/{account_id}'], function () {
Route::get('detail', function ($account_id) { /* ... */ });
Route::get('settings', function ($account_id) { /* ... */ });
});
```

Redirecciones

Como respuesta a una petición también podemos devolver una redirección. Esta opción



será interesante cuando, por ejemplo, el usuario no esté logueado y lo queramos redirigir al formulario de login, o cuando se produzca un error en la validación de una petición y queramos redirigir a otra ruta.

Para esto simplemente tenemos que utilizar el método `redirect` indicando como parámetro la ruta a redireccionar, por ejemplo:

```
return redirect('user/login');
```

O si queremos volver a la ruta anterior simplemente podemos usar el método `back`:

```
return back();
```

Redirección a una acción de un controlador

También podemos redirigir a un método de un controlador mediante el método `action` de la forma:

```
return redirect()->action([HomeController::class, 'index']);
```

Si queremos añadir parámetros para la llamada al método del controlador tenemos que añadirlos pasando un array como segundo parámetro:

```
return redirect()->action([UserController::class, 'profile'], [1]);
```

Redirección con los valores de la petición

Las redirecciones se suelen utilizar tras obtener algún error en la validación de un formulario o tras procesar algunos parámetros de entrada. En este caso, para que al mostrar el formulario con los errores producidos podamos añadir los datos que había escrito el usuario tendremos que volver a enviar los valores enviados con la petición usando el método `withInput()`

```
return redirect('form')->withInput();  
  
// O para reenviar los datos de entrada excepto algunos:  
return redirect('form')->withInput($request->except('password'));
```

Este método también lo podemos usar con la función `back` o con la función `action`:

```
return back()->withInput();  
  
return redirect()->action([HomeController::class, 'index']);  
->withInput();
```

Formularios

La última versión de Laravel no incluye ninguna utilidad para la generación de formularios. En esta sección vamos a repasar brevemente como crear un formulario usando etiquetas de HTML, los distintos elementos o *inputs* que podemos utilizar, además también veremos cómo conectar el envío de un formulario con un controlador, como protegernos



de ataques CSRF y algunas cuestiones más.

Crear formularios

Para abrir y cerrar un formulario que apunte a la URL actual y utilice el método POST tenemos que usar las siguientes etiquetas HTML:

```
<form method="POST">
...
</form>
```

Si queremos cambiar la URL de envío de datos podemos utilizar el atributo

`action` de la forma:

```
<form action="{{ url('foo/bar') }}" method="POST">
...
</form>
```

La función `url` generará la dirección a la ruta indicada. Además también podemos usar la función `action` para indicar directamente el método de un controlador a utilizar, por ejemplo:

```
action([HomeController::class, 'getindex']);
```

Como hemos visto anteriormente, en Laravel podemos definir distintas acciones para procesar peticiones realizadas a una misma ruta pero usando un método distinto (GET, POST, PUT, DELETE). Por ejemplo, podemos definir la ruta "user" de tipo GET para que nos devuelva la página con el formulario para crear un usuario, y por otro lado definir la ruta "user" de tipo POST para procesar el envío del formulario. De esta forma cada ruta apuntará a un método distinto de un controlador y nos facilitará la separación del código.

HTML solo permite el uso de formularios de tipo GET o POST. Si queremos enviar un formulario usando otros de los métodos (o verbos) definidos en el protocolo REST, como son PUT, PATCH o DELETE, tendremos que añadir un campo oculto para indicarlo. Laravel establece el uso del nombre "`_method`" para indicar el método a usar, por ejemplo:

```
<form action="/foo/bar" method="POST">
<input type="hidden" name="_method" value="PUT">
...
</form>
```

Laravel se encargará de recoger el valor de dicho campo y de procesarlo como una petición tipo PUT (o la que indiquemos). Además, para facilitar más la definición de este

tipo de formularios ha añadido la función `method_field` que directamente creará este campo oculto:

```
<form action="/foo/bar" method="POST">

{{ method_field('PUT') }}

...

</form>
```

Protección contra CSRF

El CSRF (del inglés *Cross-site request forgery* o falsificación de petición en sitios cruzados) es un tipo de exploit malicioso de un sitio web en el que comandos no autorizados son transmitidos por un usuario en el cual el sitio web confía.

Laravel proporciona una forma fácil de protegernos de este tipo de ataques. Simplemente tendremos que llamar al método `csrf_field` después de abrir el formulario, igual que vimos en la sección anterior, este método añadirá un campo oculto ya configurado con los valores necesarios. A continuación se incluye un ejemplo de uso:

```
<form action="/foo/bar" method="POST">

{{ csrf_field() }}

...

</form>
```

Elementos de un formulario

A continuación vamos a ver los diferentes elementos que podemos añadir a un formulario. En todos los tipos de campos en los que tengamos que recoger datos es importante añadir sus atributos `name` e `id`, ya que nos servirán después para recoger los valores rellenados por el usuario.

Campos de texto

Para crear un campo de texto usamos la etiqueta de HTML `input`, para la cual tenemos que indicar el tipo `text` y su nombre e identificador de la forma:

```
<input type="text" name="nombre" id="nombre">
```

En este ejemplo hemos creado un campo de texto vacío cuyo nombre e identificador es "nombre". El atributo `name` indica el nombre de variable donde se guardará el texto introducido por el usuario y que después utilizaremos desde el controlador para acceder al valor.

Si queremos podemos especificar un valor por defecto usando el atributo `value`:

```
<input type="text" name="nombre" id="nombre" value="Texto inicial">
```

Desde una vista con *Blade* podemos asignar el contenido de una variable (en el ejemplo `$nombre`) para que aparezca el campo de texto con dicho valor. Esta opción es muy útil para crear formularios en los que tenemos que editar un contenido ya existente, como por ejemplo editar los datos de usuario. A continuación se muestra un ejemplo:

```
<input type="text" name="nombre" id="nombre" value="{{ $nombre }}">
```

Para mostrar los valores introducidos en una petición anterior podemos usar el método `old`, el cual recuperará las variables almacenadas en la petición anterior. Por ejemplo, imaginad que creáis un formulario para el registro de usuarios y al enviar el formulario comprobáis que el usuario introducido está repetido. En ese caso se tendría que volver a mostrar el formulario con los datos introducidos y marcar dicho campo como erróneo. Para esto, después de comprobar que hay un error en el controlador, habría que realizar una redirección a la página anterior añadiendo la entrada como ya vimos con `withInput()`, por ejemplo: `return back()->withInput();`. El método `withInput()` añade todas las variables de entrada a la sesión, y esto nos permite recuperarlas después de la forma:

```
<input type="text" name="nombre" id="nombre" value="{{ old('nombre') }}">
```

Más adelante, cuando veamos como recoger los datos de entrada revisaremos el proceso completo para procesar un formulario.

Más campos tipo *input*

Utilizando la etiqueta `input` podemos crear más tipos de campos como contraseñas o campos ocultos:

```
<input type="password" name="password" id="password">
<input type="hidden" name="oculto" value="valor">
```

Los campos para contraseñas lo único que hacen es ocultar las letras escritas.

Los campos ocultos se suelen utilizar para almacenar opciones o valores que se desean enviar junto con los datos del formulario pero que no se tienen que mostrar al usuario. En las secciones anteriores ya hemos visto que Laravel lo utiliza internamente para almacenar un *hash* o código para la protección contra ataques tipo CSRF y que también lo utiliza para indicar si el tipo de envío del formulario es distinto de POST o GET. Además nosotros lo podemos utilizar para almacenar cualquier valor que después queramos recoger justo con los datos del formulario.

También podemos crear otro tipo de *inputs* como *email*, *number*, *tel*, etc. (podéis consultar la lista de tipos permitidos aquí:



http://www.w3schools.com/html/html_form_input_types.asp).

Para definir estos campos se hace exactamente igual que para un campo de texto pero cambiando el tipo por el deseado, por ejemplo:

```
<input type="email" name="correo" id="correo">  
<input type="number" name="numero" id="numero">  
<input type="tel" name="telefono" id="telefono">
```

Textarea

Para crear un área de texto simplemente tenemos que usar la etiqueta HTML

textarea de la forma:

```
<textarea name="texto" id="texto"></textarea>
```

Esta etiqueta además permite indicar el número de filas (rows) y columnas (cols) del área de texto. Para insertar un texto o valor inicial lo tenemos que poner entre la etiqueta de apertura y la de cierre. A continuación se puede ver un ejemplo completo:

```
<textarea name="texto" id="texto" rows="4" cols="50">Texto por  
defecto</textarea>
```

Etiquetas

Las etiquetas nos permiten poner un texto asociado a un campo de un formulario para indicar el tipo de contenido que se espera en dicho campo. Por ejemplo añadir el texto "Nombre" antes de un input tipo texto donde el usuario tendrá que escribir su nombre.

Para crear una etiqueta tenemos que usar el tag "label" de HTML:

```
<label for="nombre">Nombre</label>
```

Donde el atributo `for` se utiliza para especificar el identificador del campo relacionado con la etiqueta. De esta forma, al pulsar sobre la etiqueta se marcará automáticamente el campo relacionado. A continuación se muestra un ejemplo completo:

```
<label for="correo">Correo electrónico:</label>  
<input type="email" name="correo" id="correo">
```

Checkbox y Radio buttons

Para crear campos tipo *checkbox* o tipo *radio button* tenemos que utilizar también la etiqueta `input`, pero indicando el tipo `checkbox` o `radio` respectivamente. Por ejemplo, para crear un *checkbox* para aceptar los términos escribiríamos:

```
<label for="terms">Aceptar términos</label>  
<input type="checkbox" name="terms" id="terms" value="1">
```

En este caso, al enviar el formulario, si el usuario marca la casilla nos llegaría la variable con nombre `terms` con valor 1. En caso de que no marque la casilla no llegaría nada, ni

siquiera la variable vacía.

Para crear una lista de *checkbox* o de *radio button* es importante que todos tengan el **mismo nombre** (para la propiedad name). De esta forma los valores devueltos estarán agrupados en esa variable, y además, el *radio button* funcionará correctamente: al apretar sobre una opción se desmarcará la que esté seleccionada en dicho grupo (entre todos los que tengan el mismo nombre). Por ejemplo:

```
<label for="color">Elige tu color favorito:</label>
<br>
<input type="radio" name="color" id="color" value="rojo">Rojo<br>
<input type="radio" name="color" id="color" value="azul">Azul<br>
<input type="radio" name="color" id="color"
value="amarillo">Amarillo<br>
<input type="radio" name="color" id="color" value="verde">Verde<br>
```

Además podemos añadir el atributo `checked` para marcar una opción por defecto:

```
<label for="clase">Clase:</label>
<input type="radio" name="clase" id="clase" value="turista"
checked>Turista<br>
<input type="radio" name="clase" id="clase"
value="preferente">Preferente<br>
```

Ficheros

Para generar un campo para subir ficheros utilizamos también la etiqueta *input*

indicando en su tipo el valor `file`, por ejemplo:

```
<label for="imagen">Sube la imagen:</label>
<input type="file" name="imagen" id="imagen">
```

Para enviar ficheros la etiqueta de apertura del formulario tiene que cumplir dos requisitos importantes:

El método de envío tiene que ser POST o PUT.

Tenemos que añadir el atributo `enctype="multipart/form-data"` para indicar la codificación.

A continuación se incluye un ejemplo completo:

```
<form enctype="multipart/form-data" method="post">
<label for="imagen">Sube la imagen:</label>
<input type="file" name="imagen" id="imagen">
</form>
```

Listas desplegables

Para crear una lista desplegable utilizamos la etiqueta HTML `select`. Las opciones la indicaremos entre la etiqueta de apertura y cierre usando elementos `option`, de la forma:



```
<select name="marca">
<option value="volvo">Volvo</option>
<option value="saab">Saab</option>
<option value="mercedes">Mercedes</option>
<option value="audi">Audi</option>
</select>
```

En el ejemplo anterior se creará una lista desplegable con cuatro opciones. Al enviar el formulario el valor seleccionado nos llegará en la variable `marca`. Además, para elegir una opción por defecto podemos utilizar el atributo `selected`, por ejemplo:

```
<label for="talla">Elige la talla:</label>
<select name="talla" id="talla">
<option value="XS">XS</option>
<option value="S">S</option>
<option value="M" selected>M</option>
<option value="L">L</option>
<option value="XL">XL</option>
</select>
```

Botones

Por último vamos a ver como añadir botones a un formulario. En un formulario podremos añadir tres tipos distintos de botones:

`submit` para enviar el formulario,

`reset` para restablecer o borrar los valores introducidos y

`button` para crear botones normales para realizar otro tipo de acciones (como volver a la página anterior).

A continuación se incluyen ejemplo de cada uno de ellos:

```
<button type="submit">Enviar</button>
<button type="reset">Borrar</button>
<button type="button">Volver</button>
```